

# CREATE TABLE

---

## CREATE TABLE

*Come definire una tabella, le sue colonne e le prime regole di base.*

## Creare la struttura di una tabella

`CREATE TABLE` serve a creare una nuova tabella.

Non stai ancora inserendo dati. Stai preparando lo spazio che li conterrà: scegli il nome della tabella, le colonne e alcune regole di base.

È come preparare un modulo prima di farlo compilare. Decidi quali campi servono: nome, email, data di iscrizione.

## Un primo esempio

Questa query crea una tabella per salvare clienti:

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(150),  
  data_iscrizione DATE  
);
```

Dopo questa query, la tabella `clienti` esiste, ma è vuota. Per aggiungere righe userai `INSERT`.

## Leggiamola con calma

Dentro le parentesi trovi le colonne:

- `id` è un numero intero e identifica ogni cliente.
- `nome` è testo fino a 100 caratteri e non può mancare.

- `email` è testo fino a 150 caratteri.
- `data_iscrizione` contiene una data.

La tabella, tradotta in italiano, dice:

*“Ogni cliente ha un id, un nome, forse un'email e forse una data di iscrizione.”*

Quando una tabella si riesce a leggere così, sei sulla strada giusta.

## La chiave primaria

**PRIMARY KEY** indica la **chiave primaria**, cioè il valore che identifica una riga senza confonderla con altre.

È come il numero di tessera di una palestra. Due persone possono chiamarsi entrambe Luca, ma non dovrebbero avere lo stesso numero di tessera.

Per questo spesso si usa una colonna `id` : non descrive la persona, la identifica.

## Colonne obbligatorie con **NOT NULL**

**NOT NULL** significa che una colonna non può restare vuota.

Nel nostro esempio:

```
nome VARCHAR(100) NOT NULL
```

vuol dire che ogni cliente deve avere un nome.

**Suggerimento:** Usa **NOT NULL** per i dati davvero necessari. Se un'informazione può mancare nella vita reale, non renderla obbligatoria solo per abitudine.

## Tipi e regole

Quando crei una colonna, scegli almeno due cose:

- il **tipo**, cioè che forma avrà il dato
- le **regole**, cioè cosa il database deve controllare

Per esempio:

```
prezzo DECIMAL(10, 2) NOT NULL
```

Qui `DECIMAL(10, 2)` dice che `prezzo` è un numero decimale. `NOT NULL` dice che il prezzo deve esserci.

Il database non conserva soltanto i dati. Ti aiuta anche a tenerli ordinati e coerenti.

## Scegliere colonne chiare

All'inizio può venire voglia di creare colonne generiche come `info` o `dati`.

Meglio evitarlo. Una tabella diventa più utile quando ogni colonna ha un compito chiaro.

```
CREATE TABLE prodotti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  prezzo DECIMAL(10, 2) NOT NULL  
);
```

Qui capisci subito cosa contiene ogni colonna. Non devi indovinare.

## Prima di creare una tabella

Fai queste domande:

- che cosa rappresenta ogni riga?
- quale colonna identifica una riga?
- quali dati sono obbligatori?
- quali dati possono mancare?
- i tipi scelti sono adatti ai valori reali?

Non serve progettare tutto perfettamente al primo colpo. Però queste domande ti evitano molte tabelle confuse.

## Riepilogo rapido

- `CREATE TABLE` crea la struttura di una tabella.

- Ogni colonna ha un nome e un tipo.
- **PRIMARY KEY** identifica ogni riga.
- **NOT NULL** rende una colonna obbligatoria.
- Dopo aver creato la tabella, puoi aggiungere dati con **INSERT** .